NOTE One of the design principles of Next.js is a configuration-free philosophy. It abstracts essential tooling functionalities like bundling, compiling, and optimization without writing a single line of code. That does not mean that all configuration management is bad; it just makes it easier to onboard new developers to the platform.

2.8.3 Environment variables in Next.js

Next.js handles environment variables differently from Express.js. You don't need to use the `dotenv` package as we did before. Instead, use an .env.local or .env file to store your environment variables for your local example. Next.js will then load all variables that are automatically loaded into `process.env`:

```
OPENAI_API_KEY=<OPENAI_SECRET_KEY>
GEMINI_API_KEY= <GEMINI_SECRET_KEY>
```

You'll access these variables in your server-side code (e.g., API routes, `getServerSideProps`) using `process.env.OPENAI_API_KEY`. This keeps your sensitive API keys secure and prevents them from being sent to the user's browser.

However, if you have nonsensitive configuration data that needs to be available in the browser (client-side code), Next.js provides a specific way to "expose" these. To make an environment variable accessible in the browser, you need to prefix its name with `NEXT_PUBLIC_`, for instance:

```
NEXT_PUBLIC_GOOGLE_ANALYTICS_ID=ga_abc
```

It's critical to understand that any variable prefixed with `NEXT_PUBLIC_` will be embedded directly into the client-side JavaScript bundle. This means it will be visible to anyone inspecting the website's source code in their browser. Therefore, you should never use `NEXT_PUBLIC_` for sensitive information like API keys, database credentials, or any other secrets. This feature is strictly for public, nonsensitive configuration values that your frontend code needs to function, such as Google Analytics IDs, public API endpoints, or feature flags.

2.8.4 Route groups

Let's consider what the wrapped parentheses in folder names do in Next.js. The (chat) folder we saw earlier acts as a route group. This means it groups related files for the chat functionality but excludes the folder name itself from the URL path. Inside the route group, we might put more route groups if we include a page.js at some point that Next.js needs to resolve to a page. Why use route groups? There are several advantages:

- *Better code organization*—Grouping related files (layout, page, and potentially API routes) under a namespace keeps your project structure clean and